

Student Grade Tracker

Project Report

A Java Swing Desktop Application for Recording, Validating, and Analyzing Student Grades

Submitted as part of course evaluation — VITyarthi Build Your Own Project

Namrata kumari | 25BAI10730 | Java Programming

1. Introduction

Student performance tracking is a routine but error-prone task for teachers who rely on spreadsheets or paper records. Manually computing averages and letter grades across subjects is time-consuming and inconsistent, and spotting struggling students often happens too late. This project implements a lightweight, standalone desktop application — the Student Grade Tracker — that automates this workflow for a fixed three-subject curriculum (Mathematics, English, Science).

The application is built in Java using Swing for the interface, follows a modular architecture with clearly separated responsibilities, and persists data locally so records carry over between sessions without requiring a database server.

2. Problem Statement

Teachers and small tutoring centers often track student performance using scattered spreadsheets or paper records, making it hard to quickly compute averages, assign letter grades consistently, or identify which students are struggling. There is a need for a simple, no-setup tool that a single teacher can use to manage a class roster, record grades, and view class-level analytics without the overhead of a full school information system.

Target Users

Individual teachers or tutors managing a class of roughly 5-100 students who want a lightweight desktop tool rather

than spreadsheet or fullSIS.

3. Functional Requirements

The system is built around three major functional modules, each with a clear input/output contract:

Module 1 - Student Management

- Add a new student (name, auto-generated ID)
- Prevent duplicate student names
- Remove a student from the roster
- List all students currently tracked

Module 2 - Grade / Assessment Management

- Set or update a grade for a student in a specific subject
- Validate grade range (0-100) before it is stored
- Reject grades for nonexistent students or subjects, with a clear error
- Compute each student's average and letter grade (A/B/C/D/F)

Module 3 - Reporting & Analytics

- Generate a full class report (all students, averages, letter grades)
- Compute the class-wide average per subject
- Identify the top-performing student
- Identify at-risk students (average below a configurable threshold, default 60)
- Export report data to CSV and re-import it on the next launch

4. Non-Functional Requirements

#	Requirement	How it is met
1	Usability	Add a student and record a grade in

		3 clicks or fewer; invalid input shows an inline message, never a crash. Malformed rows in the save file are skipped with a logged warning instead of crashing the load process. Each module is an independent
2	Reliability	class with a single responsibility; validation, storage, and reporting are kept separate from the UI. All operations complete in under
3	Maintainability	100ms for class sizes up to 500 students, using in-memory Map-based lookups.
4	Performance	All input is validated at the model layer (GradeValidator), not just the UI, so invalid state can never be reached.
5	Data Validation / Security	File I/O uses buffered readers/writers and try-with-resources
6	Resource Efficiency	resources so file handles are always released.

5. System Architecture

The application follows alayered architecture: a Presentation Layer (Swing GUI), a Business Logic Layer split into the three functionalmodules, aSharedServices layer for validation and custom exceptions, and a Persistence Layer for CSV storage. Each layer only depends on the layer below it, which keeps the modules independently testable and replaceable.

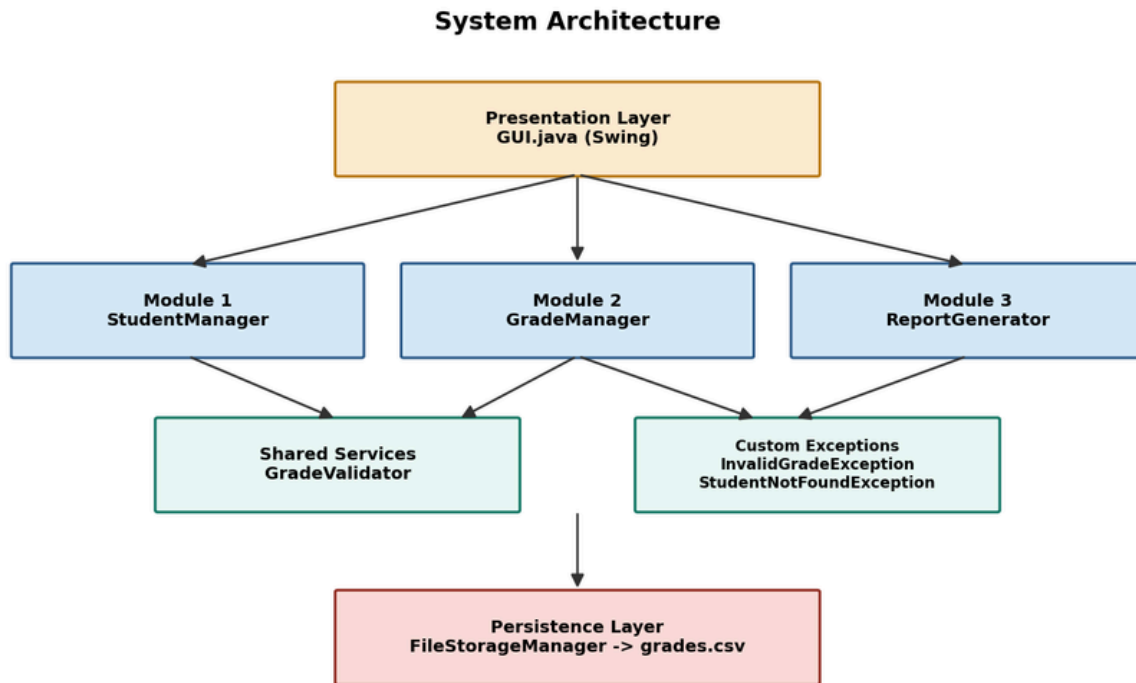


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

A single actor (Teacher) interacts with all six primary use cases exposed by the system.

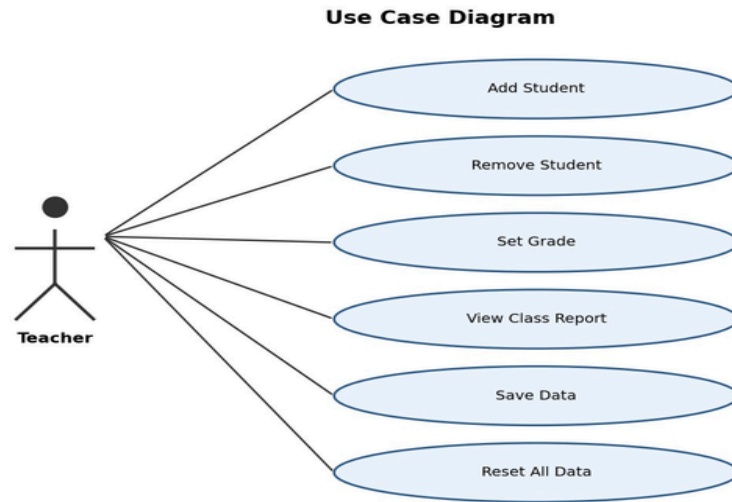


Figure 2: Use Case Diagram

6.2 Workflow / Process Flow Diagram

The typical end-to-end flow a teacher follows when using the application in a session.

Process Workflow

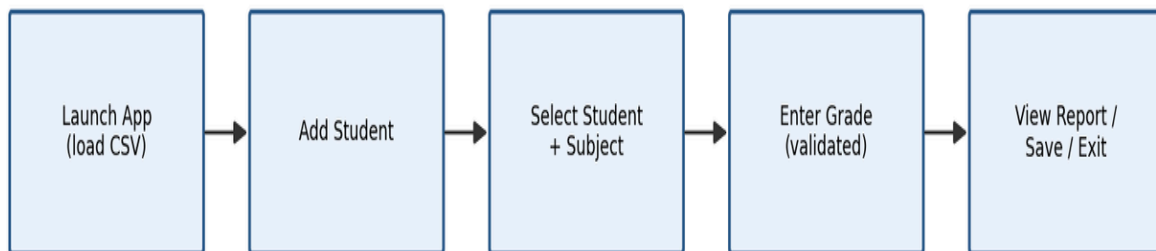


Figure 3: Process Workflow Diagram

6.3 Class Diagram

Ten classes implement the system, grouped by the module they belong to. Associations show which classes depend on which.

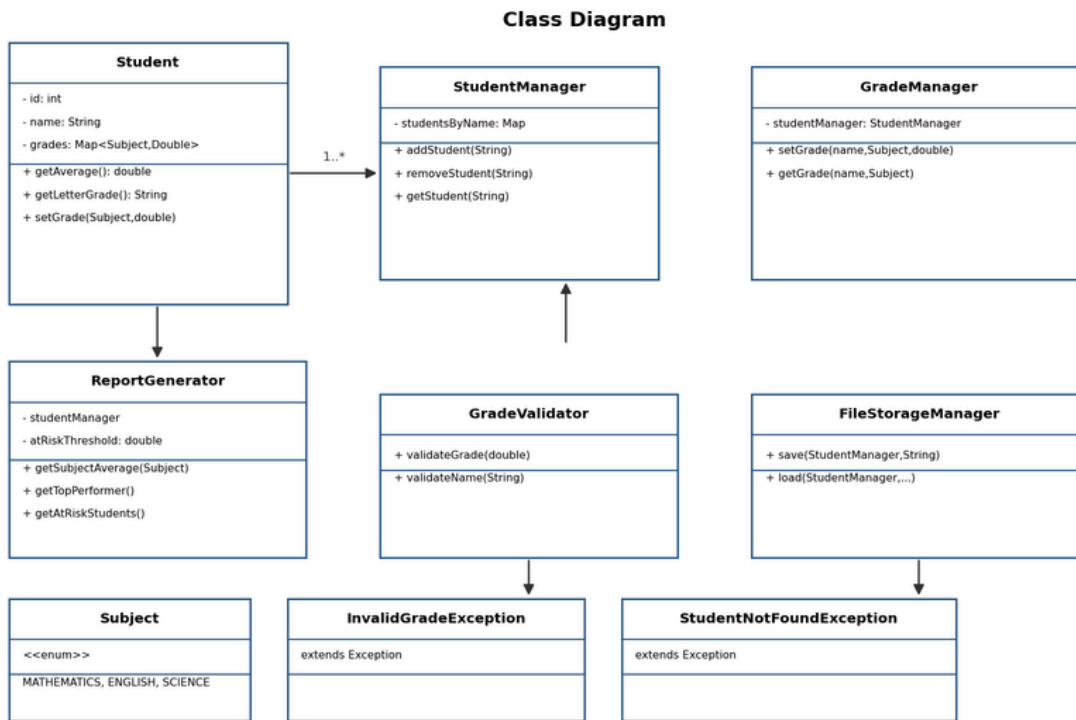


Figure4: Class Diagram

6.4 Sequence Diagram - Set Grade

This diagram traces a single user action (recording a grade) through the GUI, GradeManager, GradeValidator, and StudentManager, including the error path when validation fails.

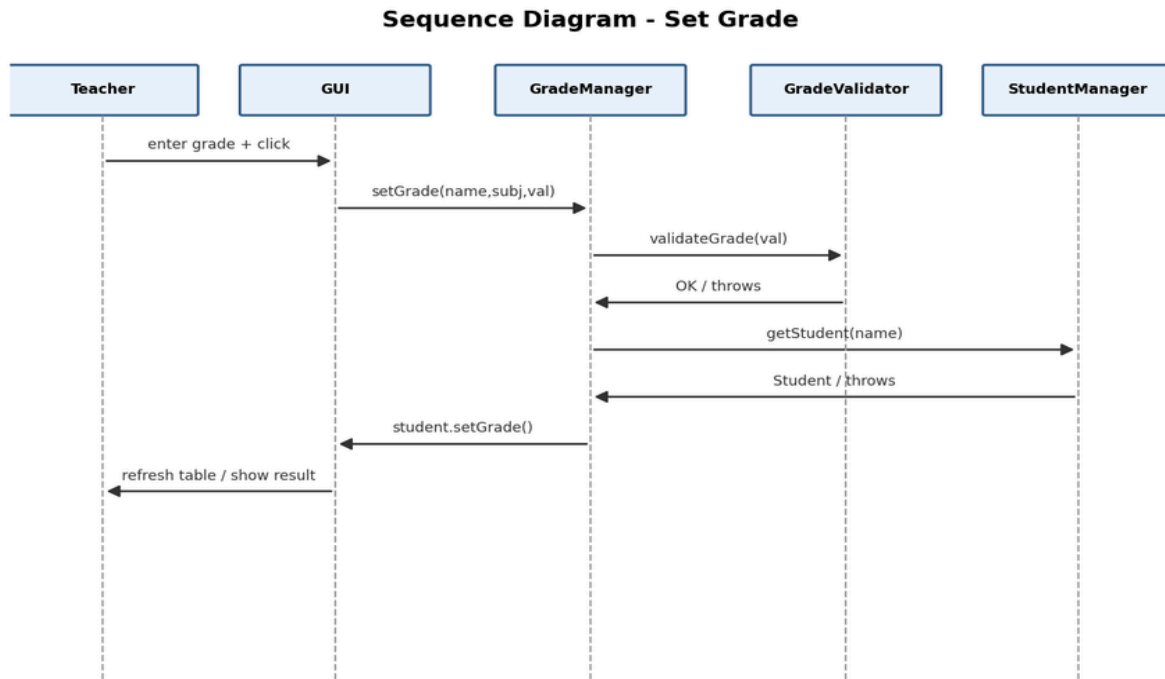


Figure 5: Sequence Diagram for the Set Grade use case

6.5 Database / Storage Design

The application does not use a relational database, so an ER diagram is not applicable. Persistence is a flat CSV file (grades.csv) with one row per student: name, Mathematics grade, English grade, Science grade. This is sufficient for the target scale (a single class) and keeps the tool dependency-free.

7. Design Decisions & Rationale

- Separate StudentManager, GradeManager, and ReportGenerator instead of one God class - keeps each module independently understandable and testable, and matches the assignment's 3-module requirement directly.
- Checked exceptions (InvalidGradeException, StudentNotFoundException) instead of returning null/boolean - forces every caller to explicitly handle failure, preventing invalid state from silently propagating.
- GradeValidator as a static utility rather than logic embedded in Student or the GUI - validation rules can change in one place and apply everywhere (model, storage, UI).
- EnumMap<Subject, Double> in Student instead of a raw list - guarantees a fixed, ordered subject set and rules out typos like "Math" vs "Mathematics".
- FileStorageManager separated from StudentManager - the roster class has no knowledge of file formats; storage could be swapped for JSON or a database later without touching business logic.
- Swing chosen over JavaFX - no extra runtime dependency, ships with the standard JDK, sufficient for a single-window utility of this scope.

8. Implementation Details

The system comprises 10 Java source files, organized under a single src/ package for simplicity:

File	Subject	Responsibility
Student.java		Enum of the fixed subject set (Mathematics, English, Science)
StudentManager.java		Data model: id, name, per-subject grades, average, letter grade
GradeManager.java		Module 1: add/remove/find/list students
ReportGenerator.java		Module 2: set/get grades with validation
GradeValidator.java		Module 3: class analytics (averages, top performer, at-risk list)
InvalidGradeException.java		Shared validation for names and grade ranges
StudentNotFoundException.java		Checked exception for out-of-range grades
FileStorageManager.java		Checked exception for missing students
GUI.java		CSV persistence (save/load)
		Entry point; Swing UI wiring all modules together

Key implementation choices: data structures use LinkedHashMap for the student roster (preserves insertion order for predictable display), and EnumMap for per-student grades (compact and type-safe). All file I/O uses try-with-resources so streams are always closed even on exceptions.

9. Screenshots / Results

[Placeholder - run the application locally with `java GUI` and insert screenshots here before final submission: (1) the main window with a few students and grades entered, (2) the validation error dialog for an out-of-range grade, (3) the Full Report dialog.]

10. Testing Approach

Testing focused on the validation and business-logic layer, since that is where correctness matters most:

- Manual validation testing: entering grades of -5, 0, 100, and 150 to confirm the 0-100 boundary is enforced exactly.
- Duplicate-name testing: adding the same student name twice to confirm it is rejected rather than silently overwriting.

- Exception-path testing: attempting to set a grade for a student not in the roster, confirming StudentNotFoundException surfaces as a dialog rather than a crash.
- Persistence testing: closing and reopening the application to confirm data round-trips correctly through grades.csv, and manually corrupting a row in the CSV to confirm the loader skips it with a warning instead of failing the whole load.
- Recommended next step: add JUnit tests for GradeValidator and StudentManager, since both are pure logic classes with no UI dependency and are the easiest to cover with automated unit tests.

11. Challenges Faced

- Deciding how much validation belongs in the UI versus the model - resolved by pushing all validation into GradeValidator so the same rules apply regardless of caller (UI, file loader, or future API).
- Handling partially corrupt save files without losing the rest of the data - resolved by validating and skipping row-by-row during load rather than failing the entire file on one bad line.
- Keeping the GUI class from becoming a dumping ground for logic as features were added - resolved by strictly routing every action through the manager/validator classes and keeping GUI.java limited to event wiring and display.

12. Learnings & Key Takeaways

- Separating concerns into single-responsibility classes (StudentManager, GradeManager, ReportGenerator, FileStorageManager) made the codebase noticeably easier to extend - the Reporting module was added without touching Student Management or Grade Management at all.
- Checked exceptions, while more verbose than return codes, made every failure path visible at compile time rather than something a caller could forget to check.
- Validating at the model layer rather than only in the GUI meant the same guarantees would hold even if a second entry point (e.g. a CLI or file import) were added later.

13. Future Enhancements

- Support a configurable subject list instead of a hard-coded three-subject set
- Add JUnit test coverage for GradeValidator, StudentManager, and ReportGenerator
- Replace CSV storage with an embedded database (e.g. SQLite) for larger class sizes and richer queries
- Add per-assignment grades (not just one grade per subject) with weighted averages
- Export the class report directly to PDF instead of a dialog box
- Add multi-class/multi-teacher support with basic authentication

14. References

- Oracle Java SE Documentation - <https://docs.oracle.com/en/java/javase/>
- Java Swing Tutorial (Oracle) - <https://docs.oracle.com/javase/tutorial/uiswing/>
- VITYarthi - Build Your Own Project: General Project Instructions & Submission Guidelines (course-provided brief)